UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias - Fundamentos de motores de videojuegos con C++

PROYECTO 01: ARCADE - SPACE INVADERS

Nombre	Elliot Kenneth Villafaña Pasten
Usuario de GitHub	Piabonnie-Dev
Repositorio	https://github.com/Piabonnie-Dev/practica02-motor
Etiqueta de entrega	v1.0-checkpoint1

1. Descripción del juego

Space Invaders es un videojuego arcade 2D desarrollado en C++20 y SDL3. El jugador controla una nave situada en la parte inferior de la ventana y puede desplazarse horizontalmente para disparar hacia un grupo de enemigos. La horda se mueve de lado a lado, desciende al tocar los bordes y aumenta su velocidad conforme disminuye la cantidad de enemigos. El escenario incluye disparos enemigos aleatorios y cuatro búnkeres destructibles (por el enemigo y por el jugador) formados por bloques geométricos.

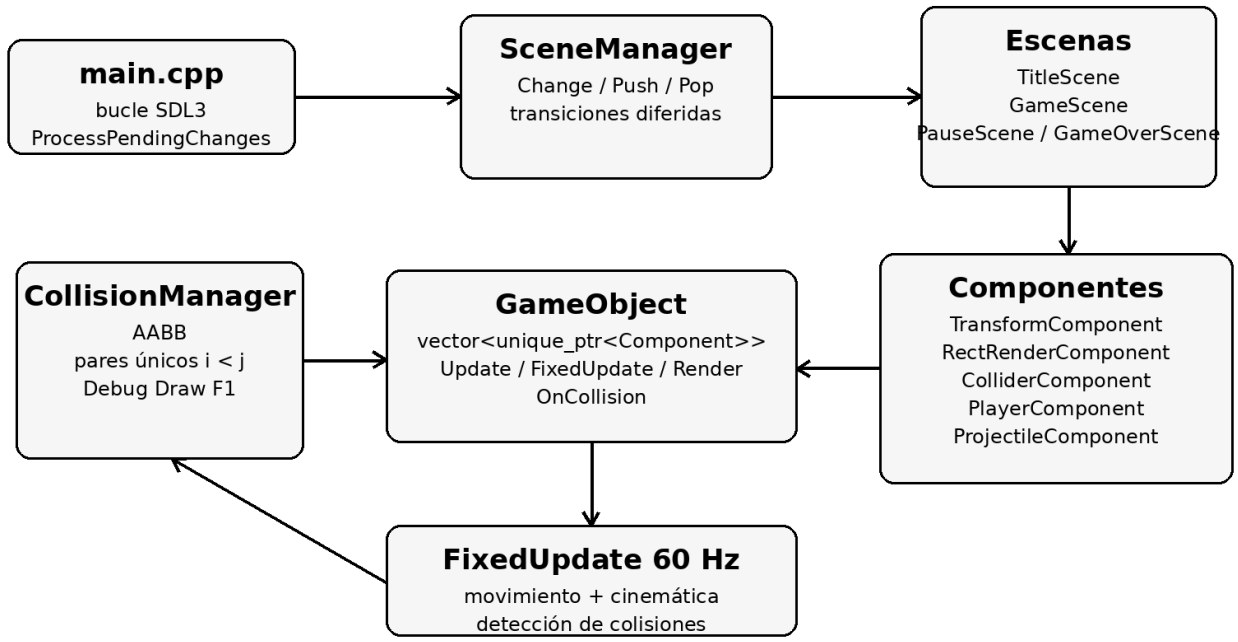
La partida comienza con 24 enemigos distribuidos en una cuadrícula de 3 filas por 8 columnas y con tres vidas para el jugador. Cada enemigo destruido otorga 100 puntos. La victoria se alcanza al eliminar a toda la horda; la derrota ocurre al agotar las tres vidas o cuando un enemigo alcanza la línea de la nave.

2. Controles y flujo de juego

Entrada	Acción
A / Flecha izquierda	Mover la nave a la izquierda
D / Flecha derecha	Mover la nave a la derecha
Espacio	Disparar / iniciar desde la pantalla de título
Enter	Iniciar desde la pantalla de título
P / Escape	Pausar o reanudar la partida
F1	Activar o desactivar Debug Draw
R	Reiniciar desde Game Over
M / Escape	Regresar al menú desde Game Over

El flujo se divide en cuatro estados: TitleScene para el inicio, GameScene para la partida, PauseScene como escena aditiva y GameOverScene para victoria o derrota. La pausa conserva el estado de la partida porque se apila sobre GameScene en lugar de destruirla.

3. Diagrama de arquitectura



4. Justificación técnica

La organización del proyecto separa las declaraciones en include/ y las implementaciones en src/. Esta división evita concentrar toda la lógica en archivos de cabecera, facilita la lectura del proyecto y mantiene main.cpp reducido a la inicialización de SDL3, el ciclo principal y la coordinación del SceneManager. Los GameObject poseen sus componentes mediante std::unique_ptr, por lo que la propiedad y liberación de memoria siguen RAII sin llamadas manuales a new o delete.

La simulación distingue Update(float dt) de FixedUpdate(float fixed_dt). Update se utiliza para lógica general, temporizadores, lectura de acciones y cambios de estado, mientras que FixedUpdate ejecuta movimiento y cinemática con un paso fijo de 1/60 s. Esto desacopla la física de la velocidad de renderizado y mantiene un comportamiento más estable entre equipos.

Las colisiones se resuelven con AABB desde CollisionManager. El administrador revisa pares únicos de entidades con iteración i < j y, cuando detecta una intersección, notifica a ambos GameObject mediante OnCollision(GameObject* other). Los proyectiles del jugador destruyen enemigos o bloques de búnker y los proyectiles enemigos destruyen búnkeres o reducen las vidas del jugador. F1 activa la representación de los colisionadores: verde cuando están libres y rojo cuando existe contacto.

SceneManager utiliza una pila de escenas y transiciones diferidas. ChangeScene sustituye la escena activa, mientras que PushScene y PopScene permiten colocar PauseScene sobre GameScene sin perder entidades, puntuación o posiciones. Las acciones pendientes se procesan al finalizar el fotograma, evitando destruir una escena mientras todavía ejecuta HandleEvent o Update.

Todo el apartado visual se construyó con primitivas geométricas de SDL3. La nave, enemigos, proyectiles, búnkeres, indicadores y pantallas utilizan rectángulos y colores simples; no se cargan texturas, spritesheets ni fuentes externas.

5. Verificación

El proyecto compila con CMake bajo C++20 y SDL3 en MSYS2 UCRT64. Se verificó el movimiento del jugador, la cadencia de disparo, el desplazamiento y descenso de la horda, el aumento de velocidad, los disparos enemigos, los búnkeres destructibles, la actualización de vidas y puntos, las condiciones de victoria y derrota, la pausa sin pérdida de estado y el Debug Draw con F1.

6. Capturas de pantalla

Las siguientes evidencias corresponden al funcionamiento del proyecto.

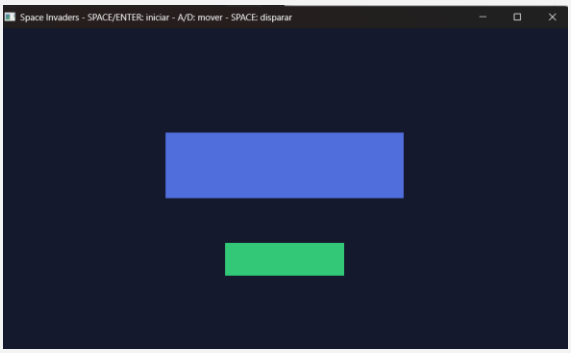


Figura 1. Pantalla de inicio.



Figura 2. Partida activa con vidas y puntuación.



Figura 3. PauseScene superpuesta sobre la partida.



Figura 4. Debug Draw activo con colisionadores visibles.

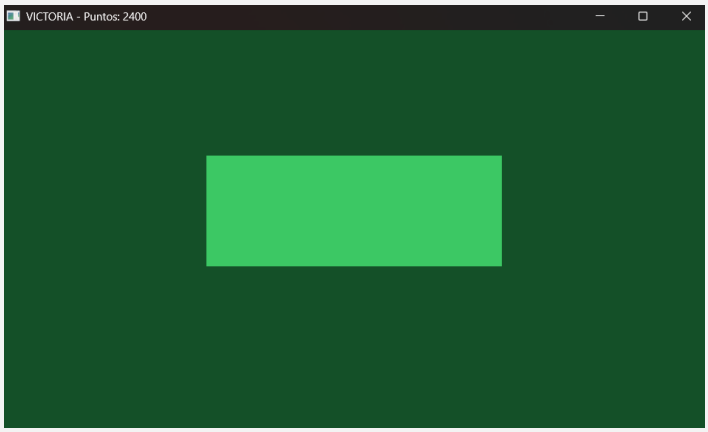


Figura 5. Pantalla de fin de partida.

7. Conclusión

El proyecto integra en una sola aplicación los subsistemas desarrollados durante las primeras prácticas: Vector2, GameObject-Component, colisiones AABB, SceneManager, FixedUpdate y gestión de memoria con std::unique_ptr. El resultado es una versión funcional de Space Invaders con reglas completas de juego, depuración visual y flujo de escenas.